

Loan Approved

```
import pandas as pd
import numpy as np

df=pd.read_csv('loan_approval_dataset.csv')

df.head()

df.info()

df.keys()

df.columns=df.columns.str.strip()

df.columns
```

drop the loan id which is not required for machine learning

```
df.drop(columns = ['loan_id'], inplace=True)
```

```
df.shape
```

```
df.head()
```

```
df.isnull().sum()
```

```
df['Assets']=df['residential_assets_value']+df['commercial_assets_value']+df['bank_asset_value']+df['luxury_assets_value']
```

```
df.head()
```

```
df.drop(columns =
['residential_assets_value','commercial_assets_value','luxury_assets_value','bank_asset_value'], inplace = True)
```

```
df.head()
```

```
df['education'].unique()
```

```
df['education']=df['education'].apply(lambda x:x.strip())
```

```
df['education'].unique()
```

```
df['education']=df['education'].replace(['Graduate','Not Graduate'],[1,0])
```

```
df.head()
```

```
df['self_employed'].unique()
```

```
df['self_employed']=df['self_employed'].apply(lambda x:x.strip())
```

```
df['self_employed']=df['self_employed'].replace(['No','Yes'],[0,1])
```

```
df.head()
```

```
df['loan_status'].unique()
```

```
df['loan_status']=df['loan_status'].apply(lambda x:x.strip())
```

```
df['loan_status']=df['loan_status'].replace(['Approved','Rejected'],[1,0])
```

```
df.head(10)
```

```
X=df.drop(columns=['loan_status'])  
y=df['loan_status']
```

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test = train_test_split(X,y,  
test_size=0.2,random_state=42)
```

```
x_train.shape, x_test.shape, y_train.shape, y_test.shape
```

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
x_train_scaled = scaler.fit_transform(x_train)
```

```
x_test_scaled = scaler.transform(x_test)
```

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(x_train_scaled, y_train)
```

```
model.score(x_test_scaled, y_test)
```

```
model.predict(x_test_scaled)
```

```
pred_data =  
pd.DataFrame([[ '2', '1', '0', '9600000', '29900000', '12', '778', '50700000']],columns=[  
'no_of_dependents', 'education', 'self_employed', 'income_annum', 'loan_amount', 'loan  
_term', 'cibil_score', 'Assets'])
```

```
pred_data = scaler.transform(pred_data)
```

```
model.predict(pred_data)
```

```
import pickle as pk
```

```
pk.dump(model,open('model.pkl','wb'))
```

```
pk.dump(scaler,open("scaler.pkl",'wb'))
```

Deploy the model

```
import streamlit as st
import pandas as pd
import pickle as pk

model=pk.load(open('model.pkl','rb'))
scaler=pk.load(open('scaler.pkl','rb'))

st.header('Loan Prediction App')

no_of_dep = st.slider('Choose No of dependents', 0, 5)
grad = st.selectbox('Choose Education',['Graduated','Not Graduated'])
self_emp = st.selectbox('Self Ememployed ?',['Yes','No'])
Annual_Income = st.slider('Choose Annual Income', 0, 10000000)
Loan_Amount = st.slider('Choose Loan Amount', 0, 10000000)
Loan_Dur = st.slider('Choose Loan Duration', 0, 20)
Cibil = st.slider('Choose Cibil Score', 0, 1000)
Assets = st.slider('Choose Assets', 0, 10000000)
```

```
if grad == 'Graduated':
    grad_s = 0
else:
    grad_s = 1

if self_emp == 'No':
    emp_s = 0
else:
    emp_s = 1

if st.button("Predict"):
    pred_data =
pd.DataFrame([[no_of_dep, grad_s, emp_s, Annual_Income, Loan_Amount, Loan_Dur, Cibil, Assets]],
              columns=['no_of_dependents', 'education', 'self_employed',
'income_annum', 'loan_amount', 'loan_term', 'cibil_score', 'Assets'])
    pred_data = scaler.transform(pred_data)
    predict = model.predict(pred_data)
    if predict[0] == 1:
        st.markdown('Loan Is Approved')
    else:
        st.markdown('Loan Is Rejected')
```